

# How to Apply Binding Attributes to Model Properties in ASP.NET Core Web API

Back to: [ASP.NET Core Web API Tutorials](#)

## How to Apply Binding Attributes to Model Properties

In this article, I will discuss **how to apply Model Binding Attributes to Model Properties in an ASP.NET Core Web API** Application with Examples, such as using FromRoute, FromQuery, and FromHeader Attributes. Please read our previous article discussing [Custom Model Binding in ASP.NET Core Web API](#) with Examples.

### Applying Model Binding Attributes to Model Properties in ASP.NET Core Web API

In ASP.NET Core Web API, **Model Binding** automatically maps incoming HTTP request data, from routes, query strings, headers, or the request body, to action method parameters or model objects.

In most cases, developers apply attributes such as [FromRoute], [FromQuery], [FromHeader], and [FromBody] **directly to action method parameters**. However, this approach can make controller methods longer, repetitive, and harder to maintain as APIs grow.

To address this, ASP.NET Core allows you to apply these same attributes **directly to model properties**. This property-level approach offers several key advantages:

- It **centralizes the Binding Logic** in the model itself.
- It **simplifies Controller Code** by reducing repetitive annotations.
- It makes the model **Self-Descriptive**, each property knows exactly where its value comes from.
- It improves **Reusability** across multiple actions or APIs that use the same data pattern.

Let's understand both approaches step-by-step with one real-time application.

### Real-World Scenario – Hotel Room Booking API

In a real-world hotel booking system, a single API call usually collects data from several parts of the HTTP request.

- The **Hotel ID** is passed through the **Route**, for example, `/api/hotels/501/book`. This helps the API identify which hotel the booking belongs to.
- The **Check-in** and **Check-out dates** are often sent as **Query String Parameters**, such as `?checkIn=2025-12-01&checkOut=2025-12-05`. These parameters work like filters that define when the guest wants to stay.

- A **User Token** is included in the **Request Header**, for example, **X-User-Token: 8a9x-12y9-abc**. This header value ensures that the request is secure and belongs to an authenticated user.
- Finally, detailed **Booking Information**, like the guest name, room type, number of guests, and additional options, is sent in the **Request Body** in JSON format.

Together, these parts of the request represent how a real-world API gathers data from multiple sources. This makes it an ideal example for understanding **Model Binding** in ASP.NET Core Web API.

## Example Using Model Binding Attributes at the Action Method Level

### Step 1. Create Models for Request Body

#### BookingDetails

Inside the **Models** folder, create a class file named **BookingDetails.cs**, and copy-paste the following code.

```
namespace ModelBindingDemo.Models
{
    public class BookingDetails
    {
        public string GuestName { get; set; } = string.Empty;
        public string RoomType { get; set; } = string.Empty;
        public int Guests { get; set; }
        public bool IncludeBreakfast { get; set; }
    }
}
```

#### ModifyBooking

Inside the **Models** folder, create a class file named **ModifyBooking.cs**, and copy-paste the following code.

```
namespace ModelBindingDemo.Models
{
    public class ModifyBooking
    {
        public int BookingId { get; set; }
        public string? UpdatedRoomType { get; set; }
        public bool? IncludeBreakfast { get; set; }
    }
}
```

### Step 2. Create Controller

Create an API Empty Controller named **HotelsController** within the **Controllers** folder, and then copy-paste the following code.

```
using Microsoft.AspNetCore.Mvc;
using ModelBindingDemo.Models;

namespace ModelBindingDemo.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class HotelsController : ControllerBase
    {
        // POST: /api/hotels/501/book
        // Header: X-User-Token: abc123
        // Body: BookingDetails JSON
        [HttpPost("{hotelId}/book")]
        public IActionResult BookRoom(
            [FromRoute] int hotelId,
            [FromQuery] DateTime checkIn,
            [FromQuery] DateTime checkOut,
            [FromHeader(Name = "X-User-Token")] string userToken,
            [FromBody] BookingDetails details)
        {
            if (string.IsNullOrEmpty(userToken))
                return Unauthorized("User token is missing.");

            var confirmation = new
            {
                HotelId = hotelId,
                checkIn,
                checkOut,
                UserToken = userToken,
                details.GuestName,
                details.RoomType,
                details.Guests,
                details.IncludeBreakfast
            };

            return Ok(confirmation);
        }

        // PUT: /api/hotels/501/booking/modify
        [HttpPut("{hotelId}/booking/modify")]
        public IActionResult ModifyBooking(
            [FromRoute] int hotelId,
            [FromHeader(Name = "X-User-Token")] string userToken,
            [FromBody] ModifyBooking modification)
    }
}
```

```

        {
            return Ok(new
            {
                Message = "Booking updated successfully.",
                HotelId = hotelId,
                userToken,
                modification.BookingId,
                modification.UpdatedRoomType,
                modification.IncludeBreakfast
            });
        }

// DELETE: /api/hotels/501/booking/cancel?bookingId=9001
[HttpDelete("{hotelId}/booking/cancel")]
public IActionResult CancelBooking(
    [FromRoute] int hotelId,
    [FromQuery] int bookingId,
    [FromHeader(Name = "X-User-Token")] string userToken)
{
    return Ok(new
    {
        Message = $"Booking {bookingId} for Hotel {hotelId} canceled
successfully.",
        userToken
    });
}
}
}

```

## Example Request (POST)

POST /api/hotels/501/book?checkIn=2025-12-01&checkOut=2025-12-05

Header: X-User-Token: 8a9x-12y9-abc

Body:

```

{
  "GuestName": "John Doe",
  "RoomType": "Deluxe",
  "Guests": 2,
  "IncludeBreakfast": true
}

```

## Example Request (PUT)

```
PUT /api/hotels/501/booking/modify
Header: X-User-Token: 8a9x-12y9-abc
Body:
{
  "BookingId": 9001,
  "UpdatedRoomType": "Suite",
  "IncludeBreakfast": false
}
```

### Example Request (DELETE)

```
DELETE /api/hotels/501/booking/cancel?bookingId=9001
Header: X-User-Token: 8a9x-12y9-abc
```

## Drawbacks of Action-Level Binding

This code works perfectly, but as your API grows, several problems appear:

1. **Too Many Parameters:** Each action method is cluttered with multiple parameters — making it hard to read and maintain.
2. **Repetition:** Attributes like `[FromHeader(Name = "X-User-Token")]` appear in every endpoint.
3. **Poor Reusability:** Adding new data fields (e.g., `PromoCode`, `SpecialRequest`) means editing all methods.
4. **Maintenance Overhead:** Updating the header or query name requires changing multiple actions.

## Example – Property-Level Model Binding in ASP.NET Core Web API

Now let's clean this up by **moving all binding logic into models**. Each property will define its own binding source.

### Step 1: Models with Property-Level Attributes

Inside the **Models** folder, create a class file named **BookingRequest.cs**, and copy-paste the following code.

```
using Microsoft.AspNetCore.Mvc;
namespace ModelBindingDemo.Models
{
    public class BookingRequest
    {
        [FromRoute(Name = "hotelId")]
        public int HotelId { get; set; }

        [FromQuery]
        public DateTime CheckIn { get; set; }
    }
}
```

```

        [FromQuery]
        public DateTime CheckOut { get; set; }

        [FromHeader(Name = "X-User-Token")]
        public string UserToken { get; set; } = string.Empty;

        [FromBody]
        public BookingDetails Details { get; set; } = new BookingDetails();
    }

    public class ModifyBookingRequest
    {
        [FromRoute(Name = "hotelId")]
        public int HotelId { get; set; }

        [FromHeader(Name = "X-User-Token")]
        public string UserToken { get; set; } = string.Empty;

        [FromBody]
        public ModifyBooking Modification { get; set; } = new ModifyBooking();
    }

    public class CancelBookingRequest
    {
        [FromRoute(Name = "hotelId")]
        public int HotelId { get; set; }

        [FromQuery]
        public int BookingId { get; set; }

        [FromHeader(Name = "X-User-Token")]
        public string UserToken { get; set; } = string.Empty;
    }
}

```

## Step 2. Update the Controller

Please update the HotelsController as follows:

```

using Microsoft.AspNetCore.Mvc;
using ModelBindingDemo.Models;

namespace ModelBindingDemo.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class HotelsController : ControllerBase
    {
    }
}

```

```

{
    [HttpPost("{hotelId}/book")]
    public IActionResult BookRoom(BookingRequest request)
    {
        if (string.IsNullOrEmpty(request.UserToken))
            return Unauthorized("User token is missing.");

        return Ok(new
        {
            request.HotelId,
            request.CheckIn,
            request.CheckOut,
            request.UserToken,
            request.Details.GuestName,
            request.Details.RoomType,
            request.Details.Guests,
            request.Details.IncludeBreakfast
        });
    }

    [HttpPut("{hotelId}/booking/modify")]
    public IActionResult ModifyBooking(ModifyBookingRequest request)
    {
        return Ok(new
        {
            Message = "Booking updated successfully.",
            request.HotelId,
            request.UserToken,
            request.Modification.BookingId,
            request.Modification.UpdatedRoomType,
            request.Modification.IncludeBreakfast
        });
    }

    [HttpDelete("{hotelId}/booking/cancel")]
    public IActionResult CancelBooking(CancelBookingRequest request)
    {
        return Ok(new
        {
            Message = $"Booking {request.BookingId} for Hotel {request.HotelId} canceled successfully.",
            request.UserToken
        });
    }
}
}

```

## Code Explanation

The controller is now **minimal and clean** — just one parameter per action. Each model defines exactly where its data comes from:

- [FromRoute] → hotel ID
- [FromQuery] → check-in, check-out, or booking ID
- [FromHeader] → user authentication token
- [FromBody] → request payload

Adding new fields (like PromoCode or SpecialRequest) requires updating **only the model**, not all actions. These models are reusable across other services or endpoints.

## Benefits of Property-Level Binding

- **Cleaner Controller:** Each endpoint uses a single parameter, the model.
- **Reusability:** The same models can be used across multiple endpoints.
- **Reduced Duplication:** Binding attributes are defined once per property.
- **Better Maintenance:** Adding/changing input requires only model changes.
- **Self-Documenting Models:** Each property clearly indicates where its data comes from.

The **Property-Level Model Binding** approach offers a more structured and maintainable way to design complex APIs. It avoids repetitive code, improves readability, and keeps your controller actions focused solely on business logic, not on parsing or validating request data. If you are designing APIs for **hotel booking**, **e-commerce orders**, or **payment processing**, this approach is highly recommended.

In the next article, I will discuss [Content Negotiation in ASP.NET Core Web API](#) with Examples. In this article, I will explain **how to apply model binding attributes to model properties in ASP.NET Core Web API** with examples. I hope you enjoy this article.



Dot Net Tutorials

*About the Author: Pranaya Rout*

*Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.*



# 1 thought on “How to Apply Binding Attributes to Model Properties in ASP.NET Core Web API”



SICILY\_TONG

NOVEMBER 22, 2024 AT 6:10 PM

This article is good, thank you and your team for your dedication

[Reply](#)

## Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent

Do Not Sell My Information